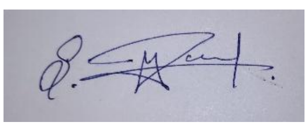
SIMATS ENGINEERING
ASSESSMENT TOOL 1: Class Test

COURSE NAME:	Cloud Computing and Big Data Analytics	**COURSE CODE:**	CSA1522

COURSE OUTCOME COVERED
CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 30**
Question Count: 10

Code of Conduct
I **MAGESH.S (192525002)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student:



Assessment Tasks Class Test

1. Analyze a multinational retail organization struggling with fragmented customer data across online and offline channels, and propose a unified big data architecture that enables real-time customer analytics and personalized recommendations.
2. Evaluate the effectiveness of distributed computing models in processing large-scale scientific research data, and justify how parallel processing improves computational efficiency and scalability compared to centralized systems.
3. Design a fraud detection framework for a digital payment platform using big data analytics, and justify how streaming data processing and predictive models improve fraud prevention accuracy.
4. Assess the challenges of storing and processing multimedia data such as video, audio, and images in a big data environment, and recommend suitable technologies and storage mechanisms for efficient management.
5. Construct a disaster recovery and business continuity strategy for a cloud-based big data platform, and justify how redundancy, replication, and automated failover mechanisms ensure operational resilience.
6. Analyze the impact of data quality issues such as inconsistency, duplication, and missing values in a large-scale analytics platform, and propose data governance strategies to improve analytical reliability.

7. Design a scalable recommendation system for an online streaming service using big data technologies, and justify how user behavior analytics and distributed processing enhance recommendation accuracy and user engagement.
8. Evaluate the role of cloud computing in modern big data ecosystems, and critically examine its advantages and limitations in terms of scalability, cost management, flexibility, and security.
9. Develop a real-time analytics framework for transportation and logistics management, and justify how big data technologies improve route optimization, fuel efficiency, and operational decisionmaking.
10. Critically analyze ethical and legal challenges associated with big data collection and analytics, and recommend organizational policies that ensure responsible data usage, regulatory compliance, and user trust.

Class Test Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Concept Accuracy	30%	Accurate and well-expressed technical answers	Mostly accurate answers	Partial accuracy	Inaccurate answers
Coverage	20%	Covers all required points	Covers most points	Limited coverage	Very poor coverage
Use of Examples	15%	Uses relevant and meaningful examples	Some relevant examples	Weak examples	No useful examples
Analytical Awareness	20%	Shows strong awareness of issues and implications	Reasonable awareness	Basic awareness	Minimal awareness
Clarity	15%	Clear and concise writing	Generally clear	Some unclear expressions	Poor clarity

Q1. Analyze a multinational retail organization struggling with fragmented customer data across online and offline channels, and propose a unified big data architecture that enables real-time customer analytics and personalized recommendations.

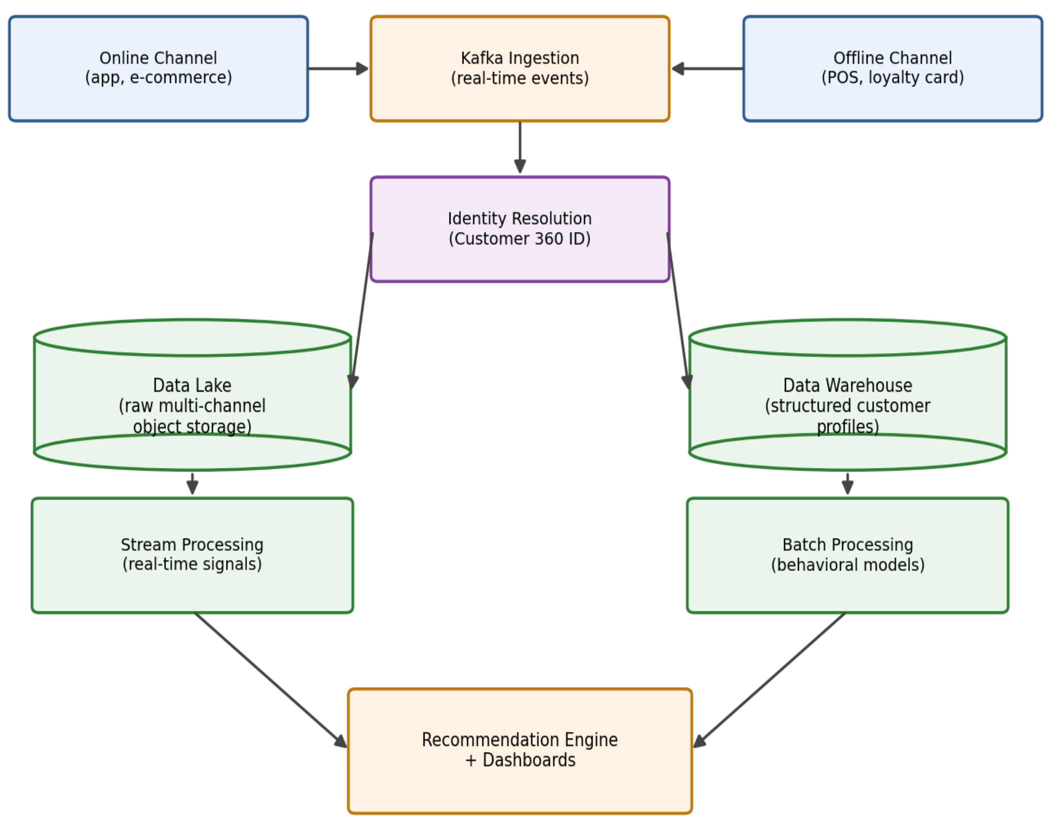
Problem analysis

Fragmented customer data typically arises because online (e-commerce, app) and offline (in-store POS, loyalty cards) channels each run on separate systems with different schemas, identifiers, and update frequencies — so no single view of a customer exists, and personalization or analytics built on any one channel misses the full picture.

Proposed architecture

- Ingestion layer: a message broker (Apache Kafka) ingests real-time events from online clickstreams, mobile app activity, and in-store POS/loyalty systems into a common pipeline.
- Unified customer ID: an identity-resolution service matches records across channels (email, phone, loyalty ID) into a single Customer 360 profile.
- Storage: a data lake (object storage) holds raw multi-channel data for flexibility, feeding a data warehouse with cleaned, structured customer profiles for fast querying.
- Processing: stream processing (Spark Streaming/Flink) computes real-time signals (e.g., "browsing category X now"), while batch processing builds deeper behavioral models overnight.
- Analytics/serving layer: a recommendation engine and real-time dashboard consume the unified profile to personalize offers across both the app and in-store kiosks.

Q1 — Unified Customer 360 Big Data Architecture



Justification

This architecture ensures a customer's online browsing and in-store purchase history feed the same recommendation model, so a customer who browses shoes online and buys in-store gets consistent, informed personalization instead of channel-siloed suggestions — directly addressing the fragmentation problem.

Q2. Evaluate the effectiveness of distributed computing models in processing large-scale scientific research data, and justify how parallel processing improves computational efficiency and scalability compared to centralized systems.

Evaluation

Scientific research data (genomics, climate simulation, particle physics) is often too large and computationally intensive for a single machine to process in reasonable time. Distributed computing models (Hadoop MapReduce, Apache Spark, HPC clusters) split both data and computation across many nodes, allowing the same workload to complete in a fraction of the time a centralized system would need.

How parallel processing improves efficiency and scalability

- Data parallelism: a large dataset (e.g., genomic sequences) is partitioned across nodes, each processing its own chunk simultaneously rather than sequentially — reducing wall-clock time roughly in proportion to the number of nodes.
- Horizontal scalability: distributed systems scale out by adding more (often commodity) nodes, whereas centralized systems scale up by buying increasingly expensive, specialized hardware with a hard ceiling on capacity.
- Fault tolerance: frameworks like Spark automatically re-run failed tasks on other nodes using data lineage, so a single node failure doesn't halt a multi-day simulation the way it would on a centralized system.
- Cost efficiency: distributed clusters can use commodity hardware and elastic cloud resources, paying only for compute during the active research run, versus a centralized supercomputer that is costly whether utilized or idle.

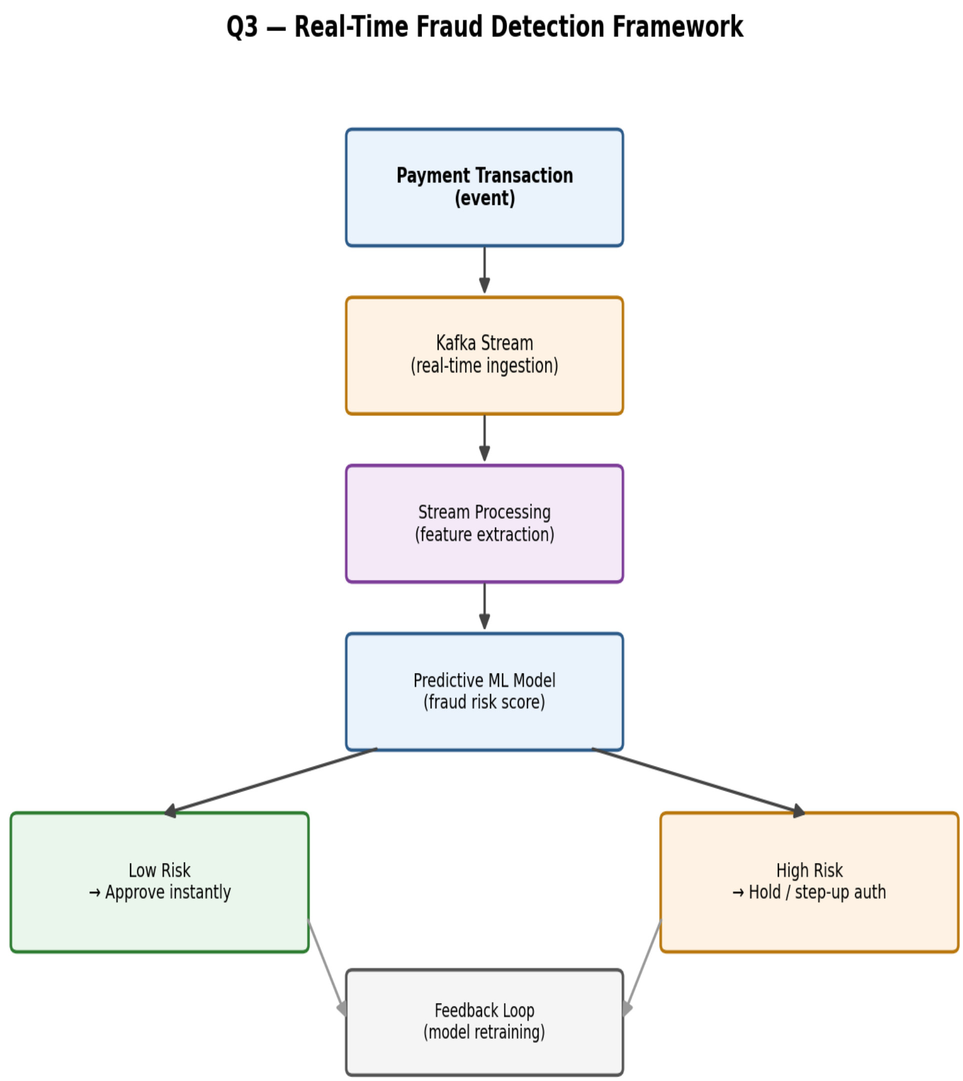
Justification with example

A genomics research project analyzing thousands of DNA sequences can partition the alignment task across hundreds of nodes using Spark, completing in hours what would take weeks on a single powerful server — demonstrating why distributed models are effective for this class of workload despite their added coordination complexity.

Q3. Design a fraud detection framework for a digital payment platform using big data analytics, and justify how streaming data processing and predictive models improve fraud prevention accuracy.

Framework design

- Real-time ingestion: every transaction event streams through a message queue (Kafka) the instant it occurs.
- Stream processing: a streaming engine (Spark Streaming/Flink) computes features on the fly — transaction velocity, unusual location, deviation from spending pattern — within milliseconds of the transaction.
- Predictive model: a machine learning model (trained on historical labeled fraud/non-fraud data, e.g., gradient boosting or a neural network) scores each transaction in real time against these features.
- Decision layer: transactions above a fraud-risk threshold are held for verification (OTP/step-up authentication) or blocked, while low-risk transactions proceed instantly.
- Feedback loop: confirmed fraud/non-fraud outcomes feed back into retraining the model periodically, improving accuracy over time.



Justification

Streaming processing is essential because fraud must be caught before the transaction settles — batch processing (analyzing transactions hours later) would only catch fraud after the money is already gone. Predictive models improve accuracy over static rule-based systems (e.g., "block if amount > $10,000") because they can detect subtle, evolving patterns — such as many small transactions in unusual sequences — that fixed rules miss, while also reducing false positives that would otherwise block legitimate customers.

Q4. Assess the challenges of storing and processing multimedia data such as video, audio, and images in a big data environment, and recommend suitable technologies and storage mechanisms for efficient management.

Challenges

- Volume: video/image files are large (unstructured), consuming massive storage compared to structured transactional data.
- Unstructured format: unlike tabular data, multimedia has no fixed schema, making traditional relational databases unsuitable for efficient storage and querying.
- Processing intensity: tasks like transcoding, object recognition, or speech-to-text on video/audio are computationally expensive and often require specialized hardware (GPUs).
- Latency requirements: streaming or real-time multimedia applications (e.g., live video) demand low-latency delivery, adding pressure on storage and network design.
- Metadata management: extracting and indexing meaningful metadata (timestamps, tags, transcripts) from multimedia content is harder than from structured records.

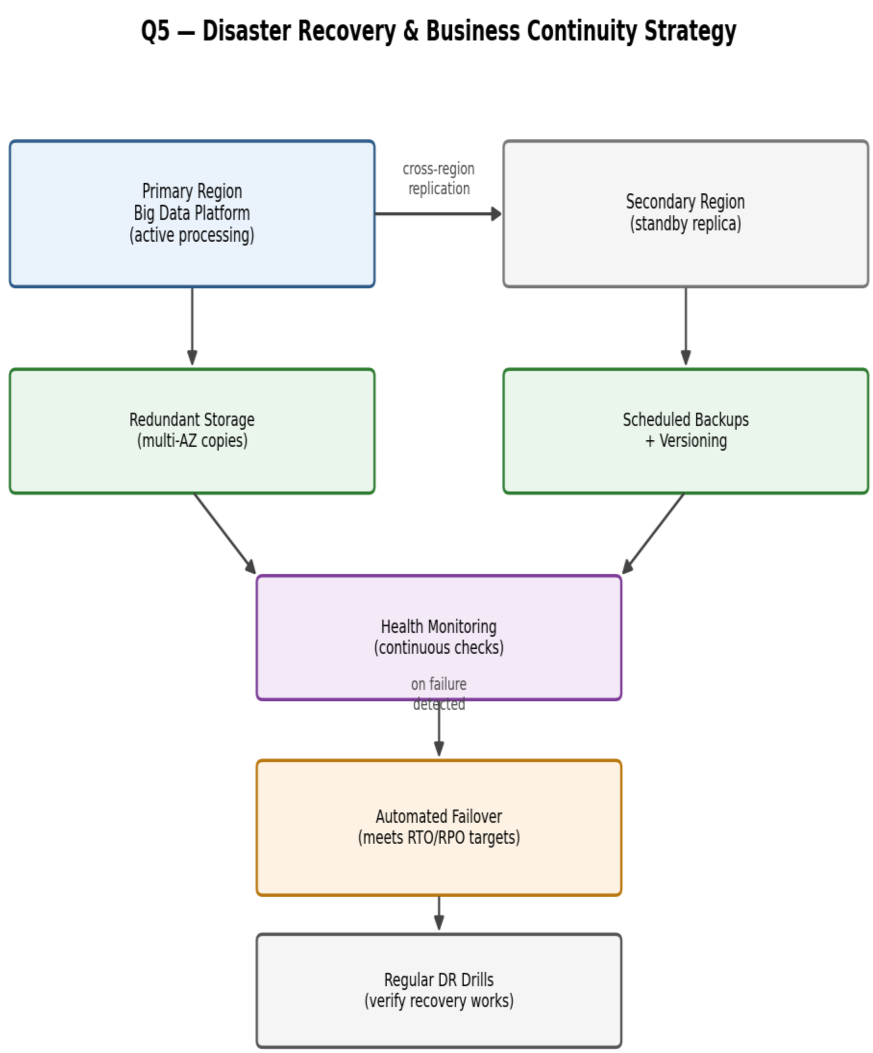
Recommended technologies and storage mechanisms

- Object storage (Amazon S3, Azure Blob Storage) for storing raw multimedia files cost-effectively at scale, since it's designed for large, unstructured binary objects.
- CDN for efficient, low-latency delivery of video/audio content to end users globally.
- Distributed processing frameworks (Apache Spark with GPU acceleration, or specialized services like AWS MediaConvert/Rekognition) for transcoding, compression, and content analysis at scale.
- NoSQL/metadata stores (e.g., a document database) to hold searchable metadata and tags separately from the bulk binary content, enabling efficient querying without scanning raw files.
- Tiered storage — hot storage for recently accessed/frequently streamed content, cold/archive storage for older or rarely accessed media, balancing cost and performance.

Q5. Construct a disaster recovery and business continuity strategy for a cloud-based big data platform, and justify how redundancy, replication, and automated failover mechanisms ensure operational resilience.

Strategy

- Data redundancy: store multiple copies of critical datasets across different storage devices and availability zones within a region, so a single hardware failure never causes data loss.
- Cross-region replication: maintain an asynchronously replicated copy of data and processing infrastructure in a geographically separate region, protecting against a full regional outage.
- Automated failover: health-check-driven failover mechanisms detect a primary region/cluster failure and automatically redirect processing and query traffic to the standby region without manual intervention.
- Backup and versioning: scheduled snapshots of the big data platform's state (metadata, configuration, processed datasets) with defined retention, enabling point-in-time recovery.
- Defined RTO/RPO: set explicit recovery time and recovery point objectives to size the redundancy investment appropriately for the platform's criticality.
- Regular DR drills: periodically test the failover process itself, not just the theoretical design, to confirm actual recovery works as expected.



Justification

Redundancy ensures no single point of failure can take the platform down; replication ensures that even a full-region disaster leaves a usable, near-current copy of the data elsewhere; and automated failover minimizes downtime by removing the delay of manual intervention during a crisis — together these three mechanisms are what let a big data platform meet strict operational resilience requirements (e.g., for a bank or telecom operator) rather than depending on manual, error-prone recovery.

Q6. Analyze the impact of data quality issues such as inconsistency, duplication, and missing values in a large-scale analytics platform, and propose data governance strategies to improve analytical reliability.

Impact of data quality issues

- Inconsistency (e.g., differing formats for the same field across source systems) leads to incorrect joins/aggregations, producing misleading analytics — such as undercounting a customer's total spend because their name is recorded differently in two systems.
- Duplication inflates counts and skews statistical measures (e.g., double-counted transactions overstate revenue), undermining trust in dashboards and reports.
- Missing values can bias models trained on the data (e.g., a fraud model trained mostly on complete records may underperform on real-world incomplete data) and can silently break aggregate calculations if not handled explicitly.
- Collectively, these issues erode decision-makers' trust in the analytics platform and can lead to costly, incorrect business decisions if not caught.

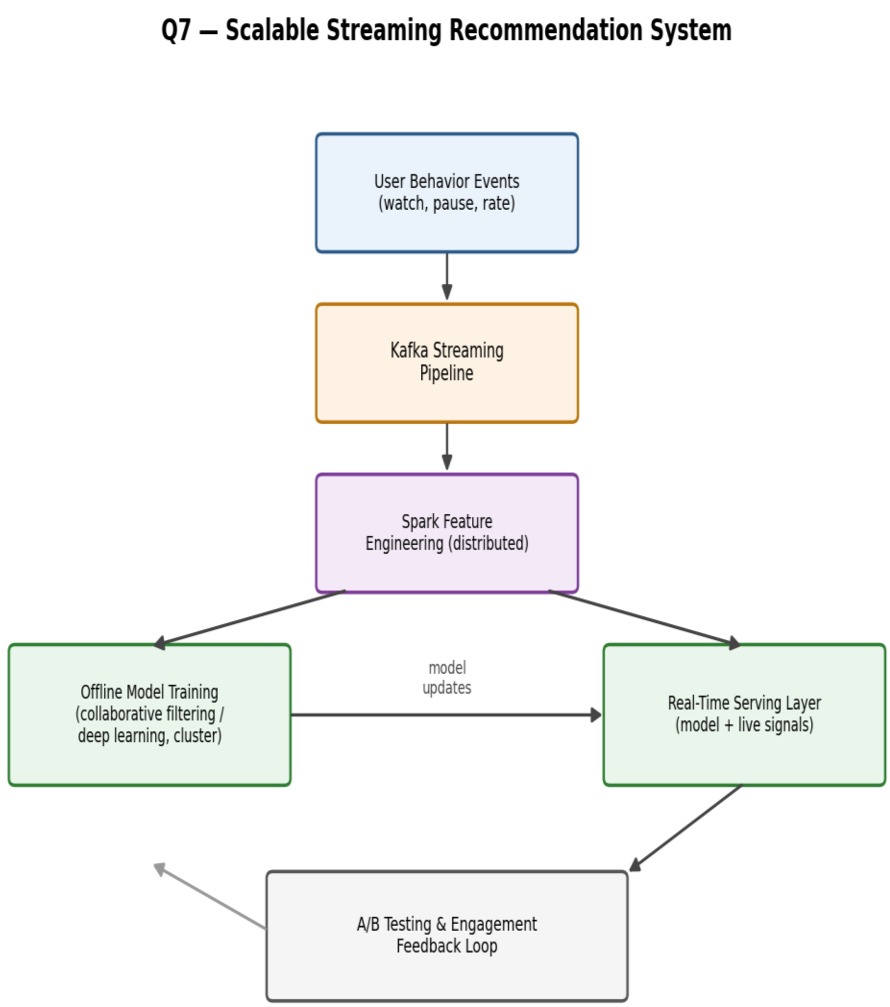
Proposed data governance strategies

- Data quality rules and validation pipelines: enforce schema validation, format standardization, and deduplication checks at ingestion time, before data enters the analytics layer.
- Master data management (MDM): maintain a single authoritative source for key entities (customers, products) that all systems reference, reducing inconsistency at the root.
- Data stewardship roles: assign clear ownership for each dataset's quality, with accountability for resolving flagged issues.
- Metadata and lineage tracking: record where each data point came from and how it was transformed, making it possible to trace and fix quality issues to their source.
- Automated data quality monitoring: dashboards that continuously score completeness, consistency, and duplication rates, alerting stewards when quality degrades below a threshold.

Q7. Design a scalable recommendation system for an online streaming service using big data technologies, and justify how user behavior analytics and distributed processing enhance recommendation accuracy and user engagement.

System design

- Data collection: capture user behavior events (watches, pauses, skips, ratings, search queries) in real time via a streaming pipeline (Kafka).
- Feature engineering: distributed processing (Spark) aggregates behavior into features — genre preferences, watch-time patterns, time-of-day habits — at scale across millions of users.
- Model training: collaborative filtering and deep-learning-based recommendation models (e.g., matrix factorization, neural embeddings) are trained on distributed clusters using historical interaction data.
- Real-time serving: a low-latency serving layer combines the offline-trained model with real-time signals (what the user just watched) to generate up-to-date recommendations on each page load.
- A/B testing framework: continuously test recommendation variants against real user engagement to iteratively improve the model.



Justification

User behavior analytics allow the system to move beyond generic, popularity-based suggestions to genuinely personalized ones — capturing subtle patterns like a user's preference for short episodes late at night versus movies on weekends. Distributed processing is essential because training and updating recommendation models across millions of users and titles would be computationally infeasible on a single machine within the time window needed to keep recommendations fresh; distributing the workload across a cluster keeps model updates timely, which directly improves both recommendation accuracy and user engagement (higher watch time, lower churn).

Q8. Evaluate the role of cloud computing in modern big data ecosystems, and critically examine its advantages and limitations in terms of scalability, cost management, flexibility, and security.

Role of cloud computing in big data

Cloud platforms provide the on-demand, elastic infrastructure (storage, compute, managed big data services like managed Spark/Hadoop, data warehouses, and ML platforms) that big data workloads need, without organizations having to build and maintain their own data centers.

Advantages

- Scalability: compute and storage can scale elastically to match data volume and processing demand, handling spikes (e.g., seasonal analytics loads) without pre-provisioning fixed hardware.
- Cost management: pay-as-you-go pricing converts large capital expenditure into variable operating expenditure, and organizations only pay for resources actually used.
- Flexibility: managed services (data lakes, serverless analytics, ML platforms) let teams adopt new big data tools quickly without lengthy procurement or setup, and multi-cloud/hybrid options avoid being locked into one specific technology stack.
- Global reach: cloud regions let organizations process and serve data close to users worldwide.

Limitations

- Cost unpredictability: at very large, sustained scale, cloud costs can exceed the cost of owned infrastructure if not carefully managed (e.g., unoptimized queries or unused resources left running).
- Security and compliance: storing sensitive data in a third-party environment raises concerns about data sovereignty, shared responsibility gaps, and potential misconfiguration-driven breaches.
- Vendor lock-in: reliance on a specific provider's proprietary big data services can make migration difficult later.
- Network dependency: performance and availability depend on network connectivity to the cloud, which can be a bottleneck for extremely latency-sensitive processing.

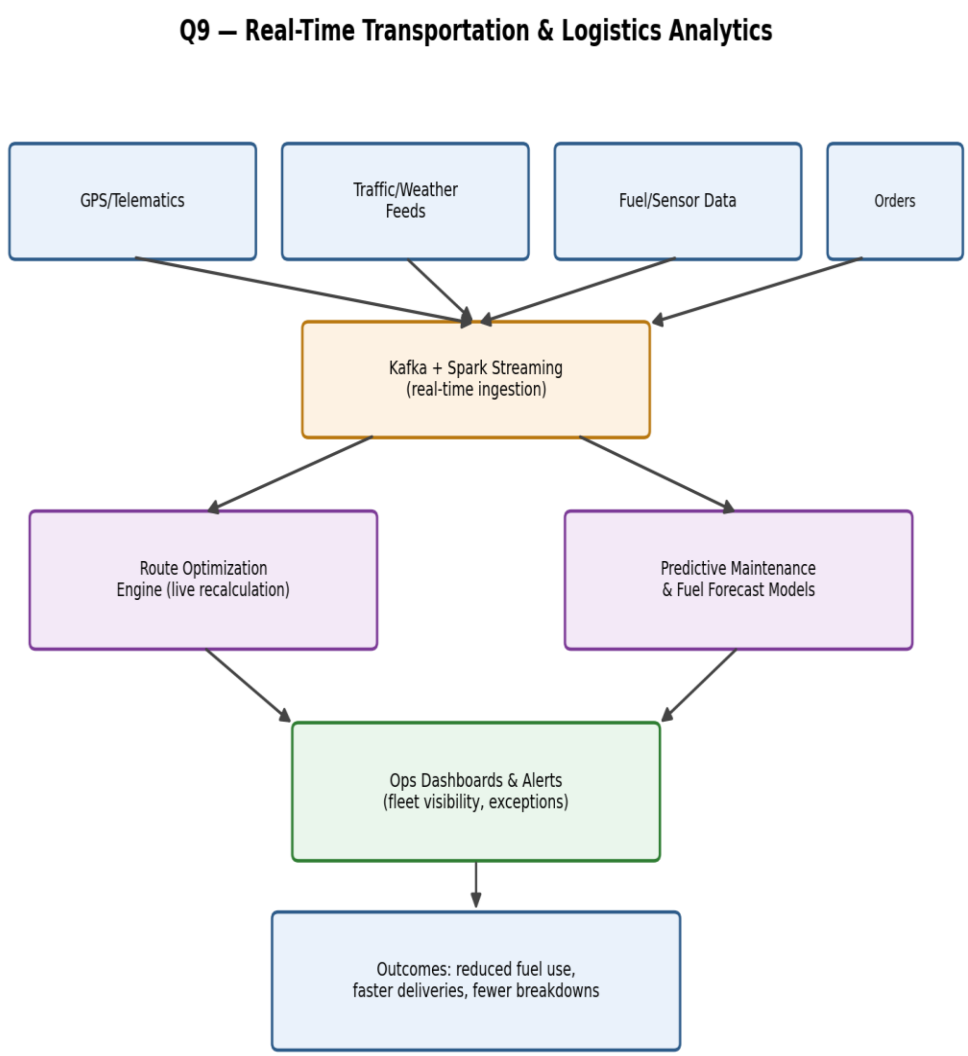
Critical view

Cloud computing has become foundational to modern big data ecosystems because it removes the capital barrier to entry for large-scale analytics, but organizations must actively manage cost optimization, security configuration, and portability to fully realize its benefits rather than assuming the cloud is automatically cheaper or safer.

Q9. Develop a real-time analytics framework for transportation and logistics management, and justify how big data technologies improve route optimization, fuel efficiency, and operational decision-making.

Framework design

- Data sources: GPS/telematics data from vehicles, traffic and weather feeds, fuel sensor data, and delivery/order data are ingested continuously.
- Streaming ingestion and processing: a stream processing pipeline (Kafka + Spark Streaming/Flink) computes real-time vehicle location, traffic conditions, and delivery ETAs as events arrive.
- Route optimization engine: a distributed algorithm (e.g., a real-time variant of vehicle routing optimization) continuously recalculates optimal routes based on live traffic and delivery priorities.
- Predictive analytics: machine learning models forecast fuel consumption patterns and predict vehicle maintenance needs from sensor data, enabling proactive scheduling.
- Dashboards and alerts: operations managers get real-time visibility into fleet status, delays, and exceptions, with automated alerts for anomalies (e.g., a vehicle significantly behind schedule).



Justification

Big data technologies enable this framework to process massive, continuous streams of GPS and sensor data from an entire fleet simultaneously — something traditional batch-oriented systems cannot do fast enough for live routing decisions. Real-time route optimization directly reduces fuel consumption and delivery time by avoiding traffic and inefficient routes as conditions change, rather than following a fixed route planned hours earlier. Predictive maintenance models reduce unplanned breakdowns, and aggregated analytics across the whole fleet give operational managers data-driven insight (e.g., which routes or drivers are consistently inefficient) that improves long-term decision-making beyond what manual dispatch logs could reveal.

Q10. Critically analyze ethical and legal challenges associated with big data collection and analytics, and recommend organizational policies that ensure responsible data usage, regulatory compliance, and user trust.

Ethical and legal challenges

- Privacy violation: large-scale data collection (often without users fully understanding what is collected) risks infringing on individual privacy, especially when data is combined across sources to build detailed personal profiles.
- Consent and transparency: many organizations collect data through long, unclear terms-of-service agreements, raising questions about whether user consent is genuinely informed.
- Bias and discrimination: analytics and ML models trained on biased historical data can perpetuate or amplify discrimination (e.g., in lending or hiring decisions) even without explicit intent.
- Regulatory compliance: laws like GDPR (EU), and country-specific data protection acts impose strict requirements on data collection, storage location, and user rights (e.g., right to be forgotten), and non-compliance carries significant legal and financial penalties.
- Data security and misuse risk: large centralized data stores are attractive targets for breaches, and internal misuse of sensitive data (e.g., unauthorized profiling) is also a real risk.

Recommended organizational policies

- Privacy-by-design: build data minimization and purpose limitation into system design from the outset, collecting only what is genuinely needed.
- Clear, accessible consent mechanisms: replace opaque legal text with plain-language explanations of what data is collected and why, with genuine opt-out options.
- Regular bias audits: periodically test analytical and ML models for discriminatory outcomes across demographic groups and correct identified biases.
- Compliance governance function: a dedicated data protection/compliance team responsible for tracking applicable regulations (GDPR, regional data protection laws) and ensuring technical controls meet them.
- Strong access control and encryption: restrict internal access to sensitive data on a need-to-know basis and encrypt data at rest and in transit to reduce breach impact.

- Transparency reporting and user rights fulfillment: provide users a straightforward way to view, correct, or delete their data, reinforcing trust and satisfying "right to be forgotten"-style regulatory requirements.